Indledning

I moderne softwaresystemer fordeles ansvar ofte mellem frontend, backend og database. Det skaber spørgsmål om, hvor forskellige typer logik bør placeres for at skabe et robust og vedligeholdelsesvenligt system.

Problemformulering

Denne synopsis vil undersøge følgende hovedspørgsmål:

> Hvordan bør forretningslogik fordeles mellem database og backend-applikation i et mindre bookingsystem? Hvilke konsekvenser har valget for vedligeholdelse, dataintegritet og systemkompleksitet?

For at besvare hovedspørgsmålet arbejdes der med følgende underspørgsmål:

1. **Hvilke arkitekturmæssige principper ligger til grund for ansvarsfordelingen mellem database og backend?**
2. **Hvordan kan et bookingsystem implementeres med henholdsvis databasecentreret og backendcentreret forretningslogik?** *Dette spørgsmål vil ikke blive besvaret direkte i løbet af denne synopsis, men vil blive besvaret til den mundtlige del af eksamenen.*
3. **Hvordan sammenlignes de 2 systemer i forhold til vedligeholdelse, dataintegritet og systemkompleksitet?**

Metode

Teoretisk del:

Indsamling af viden er sket løbende under projektet. Der er blevet læst artikler, dele af bøger og set forelæsninger. Jeg har forholdt mig kritisk over for de forskellige kilder jeg har indsamlet viden fra, og har så godt som muligt frasorteret dem jeg ikke fandt legitime.

Praktisk del:

En stor del af den viden jeg har tilegnet mig er sket under de uger jeg har udviklet 2 prototype systemer. Ideen var at udvikle 2 bookingsystemer med samme stack og sammenligne dem. Det ene system er backend centreret, det andet er database centreret. Begge systemer har samme *usecase*, *API*, *frontend* og *schema*. Forskellen ligger i hvor logikken i de 2 systemer placeres. Det backend centrerede system har primært logikken liggende i applikationslaget, som er skrevet i python. Det

database centrerede system har primært logikken liggende i MySQL som *stored procedures* og *triggers*.

Teori

Før det kan diskuteres, hvor logikken i et system bør placeres, er det en god ide at afgrænse hvad der menes når der snakkes om forretningslogik. Forretningslogik dækker over de regler og betingelser, der bestemmer, hvordan et system må behandle data. I et bookingsystem kan det eksempelvis være regler for, hvornår en booking er gyldig, hvornår en ressource er ledig, eller hvilke krav der stilles til de data, en kunde indtaster. Forretningslogik adskiller sig dermed fra præsentationlogik, som handler om hvordan information vises til brugeren, og fra ren datalagring, som handler om at gemme og hente data. Spørgsmålet er derfor helt konkret, om det giver bedst mening at lade disse regler ligge i backend koden, i databasen eller fordelt mellem begge dele.

Under min research gik det hurtigt op for mig at emnet var noget der delte vandene. I bogen “The pragmatic programmer”[1] bliver den franske datalog Bertrand Meyer nævnt. I 1986 fik han en idé. Han var utilfreds med den måde software bliver udviklet på. Han mente det var forvirrende, at der ikke var en klar ansvarsfordeling. Han mente det perfekte program er kontraktbaseret. Han udarbejder en ny metode at udvikle software på og kalder det ***design by contract***. Der er 3 grundprincipper i hans nye metode ***design by contract***.

1. ***preconditions***
2. ***postconditions***
3. ***class invariants***

Kort sagt er princippet ***preconditions*** at den der kalder funktionen i systemet “lover” at dataen der bliver videresendt til funktionen er legitim. Princippet i ***postconditions*** er at funktionen efter kaldet garanterer legitim data. ***class invariants*** princippet beskriver regler for et “objekt” der altid skal være sande.

En af de store fordele ved metoden ***design by contract*** er, at selve systemets opbygning bliver dokumentation for hvordan det skal udvikles. På database niveau kan man drage en parallel til Bertrand Meyers metode. Man har en kontrakt mellem databasen og applikationslaget i form af *constraints* på tabellers feltværdier, hvilket skaber en universel sandhed om hvilke værdier der er

forventet i systemets applikation lag. Man kan gå et skridt videre og implementere logik på database niveau i form af ***stored procedures*** - dette resulterer i en klar kontrakt om hvad og hvordan kode skal implementeres.

Man kan argumentere for at brugen af et ***Object Relational Mapping*** værktøj (***ORM***-værktøj), også til dels understøtter DBC filosofien. Implementeringen af et *ORM*-værktøj skaber umiddelbart et nyt problem for hvordan jeg tænkte mit eksperiment skulle udforme sig, da det ville skabe et yderligere abstraktionslag mellem de 2 systemer og gøre den komparative analyse svagere i min optik. Yderligere fandt jeg ud af at der er en aktiv diskussion om, hvorvidt *ORM*-værktøjer rent faktisk løser et problem. I en artikel fra universitetet i Sydney[2] udgivet af Ali Alharthy undersøger han problemet kaldet ***Object-relational impedance mismatch***. *ORM* er et værktøj der giver applikationslaget adgang til data i relationelle databaser, værktøjet lader en arbejde med data ud fra objektorienterede principper. I artiklen bygger han 2 systemer og udarbejder en komparativ analyse mellem dem. Begge systemer bygger på Java. Det ene system implementerer *hibernate*, et Java ***native object relational mapping*** værktøj. Det andet system bruger udelukkende ***Java Database Connectivity*** JDBC, som er en database driver for Java. Ali Alharthys konklusion er at kode kompleksiteten i systemet stiger efter implementeringen af et *ORM*-værktøj og fleksibiliteten af systemet falder ved brug af et *ORM*-værktøj. Der var en 4:1 ratio af kode mellem de 2 systemer. Det var altså mere tidskrævende og mindre fleksibelt at implementere et *ORM*-værktøj i kodebasen. Det skal dog siges at i hans forsøg var systemerne relativt små. Umiddelbart går hans konklusion stridt imod det der egentlig bliver lovet ved at implementere et *ORM*-værktøj som *hibernate.*

I bogen "*Designing data-intensive applications*"[3] kommer Martin Kleppmann med argumenter for og imod at implementere logic på database niveau frem for på applikationsniveau. Martin Kleppmann bruger Twitter som et af eksemplerne i bogen. Han argumenterer for, at der ikke findes en universel sandhed om, hvorvidt logikken hører til det ene sted frem for det andet. Et database centreret system kan lettere imødekomme princippet om høj datakonsistens via *triggers, stored procedures* og *constraints*. Et problem der følger med ved at implementere et system der er database centreret er at det er sværere at skalere systemet horisontalt. I et databasecentreret system bliver *horizontal scaling* vanskeligere, da databasen både fungerer som datalager og håndhæver af forretningslogik. Når logikken skal fordeles over flere database-servere, opstår der øget kompleksitet omkring synkronisering, transaktioner og konsistens. Der er 2 grundprincipper inden for database skalering. Det ene er *vertical scaling,* det andet er *horizontal scaling.*

- *vertical scaling*

  Vertical scaling er som navnet antyder, når en database skaleres vertikalt. Database serveren bliver opgraderet for at imødekomme de givne krav sat til systemet. Det kan ske i form af at opgradere RAM, CPU, Netværkskort og Lagerdrev. Der er stadig kun 1 instans af databasen, men den vokser i kapacitet, deraf navnet *vertical scaling*. I praksis er det er nemmere at implementere *vertical scaling* og vedligeholdelsesomkostningerne er lavere, da alt er samlet et sted. Problemstillinger ved *vertical scaling* er, hvis du bliver ved med at skalere. Det bliver dyrere og dyrere at opgradere din database server, og i sidste ende er der også en vis fysisk grænse for hvad 1 database server kan rumme. Hvis serveren går ned, mister du adgang til din database, der er altså *single point of failure*.

- *horizontal scaling*

Horizontal scaling løser skaleringen ved at tilføje flere separate database-servere, der arbejder sammen. Fordelene ved at skalere på denne måde er for det første og måske mest åbenlyse, at hvis en af dine servere crasher, har du stadig adgang til din database på de andre servere, givet du har replikation af data. I tilfælde af at din database er *sharded,* altså at den er opdelt på diverse servere, ville de servere der ikke er påvirket, stadig kunne tilgås. På længere sigt er det billigere at skalere på denne måde frem for *vertical scaling,* da systemet er sat op til nemt at kunne tilføje flere separate database-servere. Der er som sådan ingen øvre grænse for hvor mange database servere der kan udvides med, så du kan i teorien blive ved med at skalere dit system. Der kan tilkobles en *load balancer* som fordeler trafikken mellem dine database-servere, det optimerer responstiden på systemet. Problemstillingerne for *horizontal scaling* er, at det er langt mere komplekst at implementere i praksis, det resulterer i at startomkostningerne er højere. Hvis der er implementeret en load balancer i systemet, vil omkostningerne for at vedligeholde systemet også stige.

Martin Kleppmann argumenterer videre for, at systemer, der placerer størstedelen af logikken på applikationsniveau, ofte har lettere ved at skalere horisontalt. Problemstillinger omkring synkronisering og koordinering af logik på tværs af distribuerede database-servere reduceres, da en større del af ansvaret absorberes af applikationslaget. Jo mindre ansvar databasen har for logik og koordinering, desto lettere kan den typisk skaleres horisontalt.

[1] David Thomas & Adam Hunt, The pragmatic programmer, 1999

[2] Ali Alharthy, Design and Implementation of Business Logic Layer Object-Oriented Design

versus Relational Design, Faculty of Engineering and IT University of Technology, Sydney, 2015

[3] Martin Kleppmann, Designing data-intensive applications, 2017

Udvikling

Komparativ analyse af de 2 systemer

Med udgangspunkt i de 2 prototype systemer er det muligt at sammenligne dem med henblik på vedligeholdelse, dataintegritet og systemkompleksitet. De 2 systemer har samme usecase, API og frontend. Forskellen er at det backend centrerede system placerer størstedelen af logikken i python koden, mens det database centrerede system placerer størstedelen af logikken i MySQL i form af stored procedures og triggers. At systemerne endte med at bruge python på backend og frontend var et valg jeg tog da jeg ønskede mere erfaring med det sprog og at der var god dokumentation[4]. Grunden til at MySQL blev valgt som database var at jeg tidligere har haft gode erfaringer med databasen og der er god dokumentation[5]. MySQL er også godt integreret med terminalen, som jeg foretrækker at udvikle fra. Begge teknologier er open source, det spillede også en rolle i mit valg.

I forhold til vedligeholdelse virker det backend centrerede system[6] for mig umiddelbart lettere at arbejde med. Grunden til det er, at logikken er samlet i applikationslaget og kan læses som almindelig

programkode. Det gør det lettere at følge flowet i systemet fra request til validering og videre til databasekald. Et eksempel er funktionen **register_booking** i backend systemet, hvor man i samme flow kan se de vigtigste regler før bookingen bliver oprettet på applikationsniveau i services.py filen. Ulempen er, at reglerne i højere grad afhænger af hvordan backend koden bliver implementeret, da de ikke i samme omfang håndhæves centralt i databasen.

```
def register_booking(
    customer_id: int,
    resource_id: int,
    start_time: str,
    end_time: str,
) -> dict:
    if not customer_exists(customer_id):
        raise ValueError("Customer does not exist")

    if not resource_exists(resource_id):
        raise ValueError("Resource does not exist")

    if not resource_is_active(resource_id):
        raise ValueError("Resource is inactive")

    if start_time >= end_time:
        raise ValueError("Booking start_time must be before end_time")

    with get_connection() as conn:
        lock_resource(resource_id=resource_id, conn=conn)

        if has_overlapping_booking(
            resource_id=resource_id,
            start_time=start_time,
            end_time=end_time,
            conn=conn,
        ):
            raise ValueError(
                "Booking overlaps an existing booking for this resource"
            )

        booking = insert_booking(
            customer_id=customer_id,
            resource_id=resource_id,
            start_time=start_time,
            end_time=end_time,
            conn=conn,
        )

        conn.commit()
        return booking
```

I det database centrerede system[7] er logikken i højere grad samlet på database niveau i SQL schemaet i form af triggers og stored procedures. Et eksempel på det er proceduren **create_booking** i schema.sql.

```
CREATE PROCEDURE create_booking(
    IN p_customer_id INT,
    IN p_resource_id INT,
    IN p_start_time DATETIME,
    IN p_end_time DATETIME
 )
 BEGIN
    DECLARE v_customer_exists INT DEFAULT 0;
    DECLARE v_resource_active BOOLEAN DEFAULT NULL;
    DECLARE v_overlaps INT DEFAULT 0;

    IF p_start_time >= p_end_time THEN
       SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Booking start_time must be before end_time';
    END IF;

    START TRANSACTION;

    SELECT COUNT(*)
    INTO v_customer_exists
    FROM customers
    WHERE id = p_customer_id;

    IF v_customer_exists = 0 THEN
       SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Customer does not exist';
    END IF;

    SELECT active
    INTO v_resource_active
    FROM resources
    WHERE id = p_resource_id
    FOR UPDATE;

    IF v_resource_active IS NULL THEN
       SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Resource does not exist';
    END IF;

    IF v_resource_active = FALSE THEN
       SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Resource is inactive';
    END IF;

    SELECT COUNT(*)
    INTO v_overlaps
    FROM bookings
    WHERE resource_id = p_resource_id
      AND start_time < p_end_time
      AND end_time > p_start_time;

    IF v_overlaps > 0 THEN
       SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Booking overlaps an existing booking for this resource';
    END IF;

    INSERT INTO bookings (customer_id, resource_id, start_time, end_time)
    VALUES (p_customer_id, p_resource_id, p_start_time, p_end_time);

    COMMIT;
END
```

Det resulterer i at backend laget bliver mere simpelt, men det kan gøre flowet sværere at følge fra applikationslaget, da man skal bevæge sig fra Python koden over i databasen for at forstå hele systemets adfærd. Til gengæld kan det være en fordel hvis flere forskellige applikationer eller udviklere skal arbejde mod den samme database, fordi reglerne allerede bliver håndhævet dér. Endnu et plus kan være, at stored procedures i sig selv fungerer som dokumentation for hvordan dele af Logikken skal implementeres, på en måde der kan minde om Bertrand Meyers metode DBC. *Representational State Transfer* også kaldet *REST,* kan til dels løse den samme problematik. *REST* fungerer også som en fælles kontrakt mellem klient og backend. Hvis flere udviklere arbejder mod den samme *REST API*, bliver de i højere grad styret af de samme endpoints, dataformater og regler. Forskellen er dog, at *REST* primært håndhæver reglerne så længe al adgang går gennem backend laget. Hvis andre klienter eller systemer skriver direkte til databasen, vil disse regler kunne omgås. Stored procedures håndhæver derimod reglerne direkte i databasen og gælder derfor uanset hvilken applikation der skriver til den.

Når det kommer til dataintegritet, står det database centrerede system stærkt. Dataintegritet betyder her helt simpelt, at dataen i systemet forbliver gyldig og følger de regler der er sat op. I database systemet bliver reglerne håndhævet direkte i databasen. Et konkret eksempel er reglen om, at 2 bookinger ikke må overlappe på den samme ressource. I det database centrerede system bliver dette tjek udført direkte i stored proceduren *create_booking*, hvor databasen selv undersøger om der allerede findes en booking i det valgte tidsrum, før den nye booking bliver indsat. Hvis der findes et overlap, bliver bookingen afvist med en fejl.

I det backend centrerede system bliver den samme problematik også håndteret, men her sker det i services.py. Her bliver der først lavet et tjek med has_overlapping_booking(...), og hvis der findes en overlappende booking, bliver der kastet en fejl. Problemet er ikke, at løsningen i services.py er forkert, men at reglen i højere grad er bundet til applikationslaget. Det betyder, at reglen kun bliver håndhævet så længe oprettelsen af bookinger går gennem backend systemet.
Hvis en anden klient, et script eller et andet system skriver direkte til databasen,
vil det i backend systemet være lettere at indsætte en booking som overlapper med en eksisterende booking da reglen ikke er håndhævet direkte på database niveau. På den måde munder backend løsningen ud i lavere dataintegritet,
ikke fordi logikken er dårligere, men fordi den er placeret længere væk fra selve dataen.

Vedligeholdelse og dataintegritet viste klare forskelle mellem de 2 systemer, det er mere nuanceret i forhold til kompleksiteten. Kompleksiteten forsvinder ikke ved at flytte logikken. I det backend centrerede system er den transaktionelle kompleksitet synlig i Python koden. I *register_booking* skal udvikleren eksplicit håndtere *lock_resource()* og *conn.commit()*, hvilket giver fuld kontrol over flowet.

I det database centrerede system er *START TRANSACTION*, *FOR UPDATE* og *COMMIT* indkapslet i *create_booking* proceduren. Man kan argumenter for at der opstår kompleksitet i form af test af database-logik, det kræver ofte en aktiv database-instans, specifik testdata og håndtering af *database-state* samt transaktioner. Det kan gøre automatiseret testning mere kompleks sammenlignet med isolerede unit tests i eksempelvis Python.

Endelig peger Kleppmann på, at placering af logik i databasen kan gøre horisontal skalering vanskeligere, da databasen både skal håndtere lagring, validering og transaktioner. For et mindre bookingsystem er det måske ikke et aktuelt problem, men det viser at kompleksiteten kan ændre sig hvis systemet vokser.

Ud over de arkitektoniske overvejelser har jeg også undersøgt, om placeringen af logik påvirker systemets performance i praksis. Jeg undersøgte de 2 systemer ved at teste responstid på booking flowet. Testen målte både hvor lang tid der gik fra klienten sendte et booking request til HTTP svaret kom tilbage, og hvor lang tid der gik før bookingen kunne ses i databasen fra en separat forbindelse. Det er interessant, da det viser om placeringen af logik også har betydning for hvor hurtigt systemet reagerer i praksis.

Resultaterne for testen var at backend_heavy i gennemsnit var hurtigere end database_heavy. Backend systemet havde en gennemsnitlig responstid på 80.532 ms, mens database systemet havde 94.187 ms. I forhold til hvornår bookingen blev synlig i databasen, lå backend systemet også hurtigere med 40.843 ms mod 64.177 ms. En mulig forklaring kan være, at kald til stored procedures og transaktionsstyring giver en større samlet overhead. Det er ikke sikkert at resultatet ville være det

samme i et større system eller ved andre typer databaseoperationer, eksempelvis tungere select queries.

[4] python docs, https://docs.python.org/3/

[5] mySql docs, https://dev.mysql.com/doc/

[6] https://github.com/oscarStorm/backend_heavy

[7] https://github.com/oscarStorm/database_heavy

Konklusion

Hvilke arkitekturmæssige principper ligger til grund for ansvarsfordelingen mellem database og backend. Design by Contract var i min optik en måde at forstå problematikken på. Når logikken ligger i applikationslaget, afhænger man mere af hvordan backenden er implementeret, der er ingen klar kontrakt ud over de *constraints* der er forbundet med et *table* på database niveau og backend-API'et. Når logikken er i databasen gennem *stored procedures og triggers,* fungerer selve databasen som en kontrakt. Udover Meyers metode er Kleppmann argumenter omkring skalerbarhed med til at påvirke placeringen af logik i systemet.

Hvordan sammenlignes de 2 systemer i forhold til vedligeholdelse, dataintegritet og systemkompleksitet? Backend systemet er stærkere i forhold til vedligeholdelse, fordi logikken er lettere at læse og følge i almindelig programkode. Det database centrerede system står derimod stærkere i forhold til dataintegritet, fordi reglerne bliver håndhævet tættere på dataen. Kigger vi på systemkompleksitet, er det mere nuanceret. Kompleksiteten forsvinder ikke i det ene system frem for det andet, men antager forskellige former alt efter hvor logikken placeres.

På baggrund af afsnittet **Teori** og **Udvikling** kan det konkluderes, at der ikke findes en universel sandhed om, hvor forretningslogik bør placeres i et system generelt. Svaret afhænger af hvilke egenskaber, man ønsker at prioritere. I et mindre bookingsystem som dette, virker backend laget som det mest hensigtsmæssige sted at placere størstedelen af logikken.

Refleksion

Når projektet nu er færdigt, er der et par ting jeg godt kunne tænke mig at have gjort anderledes. Jeg ville gerne have undersøgt performance yderligere, da det kom bag på mig at det database centrerede system var langsommere end det backend tunge system. Det havde været relevant at teste om resultatet ville være det samme ved større datamængder eller andre typer forespørgsler. Jeg ville også have prioriteret mere tid til udviklingen af prototyperne, da flere af projektets vigtigste problemstillinger blev tydeligere under implementeringen. Jeg kunne også med fordel have afgrænset projektets scope yderligere.

Litteraturliste

Bøger:

David Thomas & Adam Hunt, The pragmatic programmer, 1999

Martin Kleppmann, Designing data-intensive applications, 2017

Artikler:

Ali Alharthy, Design and Implementation of Business Logic Layer Object-Oriented Design

versus Relational Design, Faculty of Engineering and IT University of Technology, Sydney, 2015

Docs:

python docs, https://docs.python.org/3/

MYSQL docs, https://dev.mysql.com/doc/

Github:

https://github.com/oscarStorm/backend_heavy

https://github.com/oscarStorm/database_heavy

Video materiale:

Code in the Database?; https://www.youtube.com/watch?v=U/sdCXu8NXvA

Application vs. Database: Who Actually Owns Your Data Model?;
https://www.youtube.com/watch?v=VzzvJSykBqs

Using constraints & partitioning to store business logic in your database;
https://www.youtube.com/watch?v=E56PNN9x_z4

Does Storing Business Logic In Your Database Help Or Hurt Performance;
https://www.youtube.com/watch?v=LUj5jKsDnlw

SQL Business Logic in .NET Apps - Ryan Bupp: OKC SQL;
https://www.youtube.com/watch?v=mWDEfI8jq0E&t=2113s